



Agencia de Habilidades
para el Futuro

<Talento
Tech/>

FRONT-END JS

Clase n° 14 | Asincronía

¡Les damos la bienvenida!



 Vamos a comenzar a grabar la clase.

Índice

1

14. Asincronía

- Asincronía
- Consumo de API REST a través de fetch
- Procesamiento de los datos
- Incluir los datos consumidos y procesados por medio de fetch en nuestro proyecto

2

15. API y Procesamiento de Datos

- Asincronía Desarrollo de un proyecto integrador que combine HTML, CSS, y JavaScript
- Consumo de API REST
- Incorporación y procesamiento de los datos en nuestro HTML
- Incorporación de buenas prácticas de accesibilidad y SEO
- Presentación del proyecto final

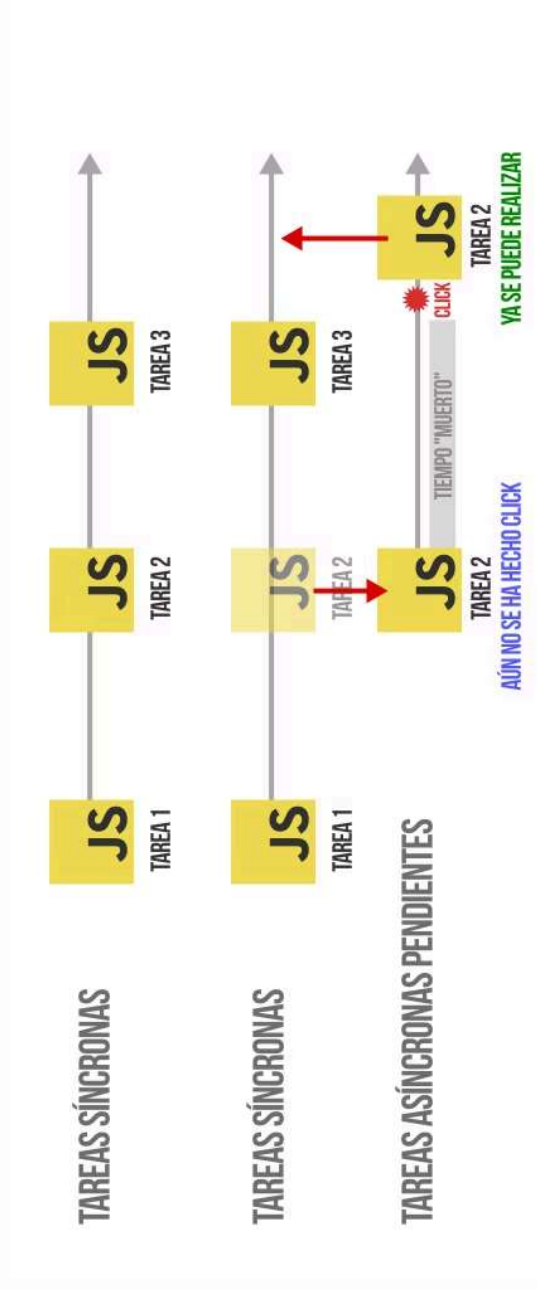
Introducción a la Asincronía

Asincronía

La asincronía permite ejecutar tareas sin bloquear el flujo principal del programa.

Beneficio

Mejora la eficiencia y la experiencia del usuario en aplicaciones web.



Fetch

Fetch: Consumir una API REST

Función para realizar solicitudes HTTP y obtener datos de forma asíncrona.

1

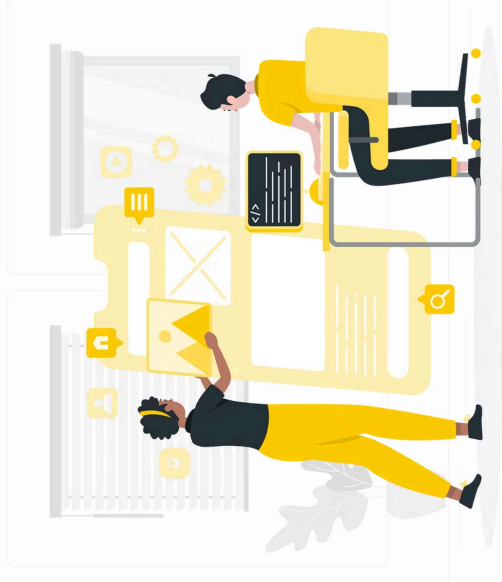
Ventaja

No interrumpe el funcionamiento de la página mientras espera la respuesta.

2

Uso básico

`fetch()` nos permite hacer peticiones a servidores externos para obtener datos.



Código básico de Fetch:

```
fetch('https://fakestoreapi.com/products')
  .then(response => response.json()) // Convertimos la respuesta a JSON
  .then(data => {
    console.log(data); // Mostramos los datos obtenidos
  })
  .catch(error => {
    console.error('Hubo un error al obtener los productos:', error);
  });
```


Promesas

Promesas en JavaScript

Cuando usás fetch, te devuelve una promesa: funciona como lo indica la palabra, es un compromiso del programa que tiene tres estados factibles: **pendiente**, **resuelta** o **rechazada**.

Resuelta (resolved)

La operación se completó con éxito.

Pendiente (pending)

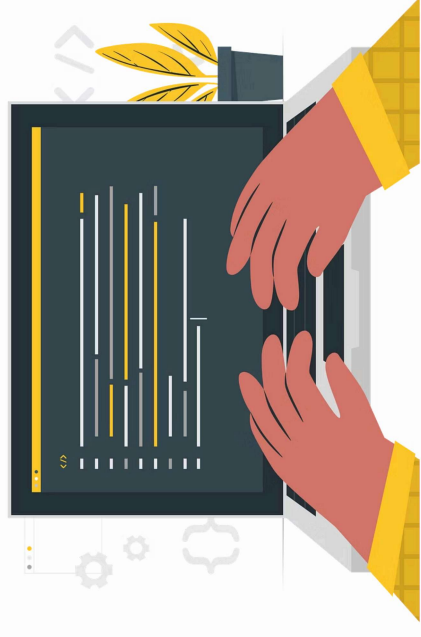
La promesa está en proceso de resolución.

Rechazada (rejected)

La operación falló.

Manejo de datos con Fetch y promesas

Manejo de Datos con Fetch y Promesas



Ahora que sabemos pedir datos, ¿qué hacemos con ellos? Supongamos que estamos creando una tienda online y queremos mostrar productos de una API externa:

En el siguiente ejemplo:

- **Recorremos** los datos de los productos que nos devolvió la API.
- Por cada producto, creamos una tarjeta (`card`) con la imagen, el nombre, el precio y un botón para agregar al carrito.
- El botón llama a la función `agregarAlCarrito`, que es la que nos va a permitir agregarlo al carrito.

Ejemplo código:

```
fetch("https://fakestoreapi.com/products")
  .then((response) => response.json())
  .then((data) => {
    const contenedor = document.getElementById("productos-container");
    data.forEach((producto) => {
      contenedor.innerHTML += `
        <div class="card">
          
          <h3>${producto.title}</h3>
          <p>Precio: ${producto.price}</p>
          <button onclick="agregarAlCarrito(${producto.id})">Añadir al
            carrito</button>
        </div>
      `;
    });
  })
  .catch((error) => console.error("Error al obtener productos:", error));
```


Manejo de Errores con Fetch

1 Usar `.catch()`

Captura errores en la promesa de fetch.

2 Mostrar mensaje de error

Informa al usuario sobre el problema ocurrido.

Ejemplo:

```
fetch("https://fakestoreapi.com/products")
  .then((response) => response.json())
  .then((data) => {
    // Mostrar productos...
  })
  .catch((error) => {
    console.error("Error al obtener productos:", error);
    alert("Hubo un problema al cargar los productos.");
  });
```



Async/Await

Async/Await: Otra Forma de Esperar

Hay otra forma de manejar la asincronía que es más sencilla y clara: **async/await**.

Async

Le decimos a la función que va a manejar tareas asíncronas.

Await

Le decimos que espere a que termine antes de seguir.

Ejemplo con Async/Await



```
async function obtenerProductos() {  
  try {  
    const response = await  
    fetch("https://fakestoreapi.com/products");  
    const data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.error("Error al obtener productos:",  
    error);  
  }  
}
```

Ejercicios Prácticos

Ejercicio Práctico #1

Obligatorio

Pasos a seguir:

Para integrar una API REST en tu proyecto de e-commerce u otro proyecto personal:

1. **Seleccionar la API:** Elige una API pública (por ejemplo, [Fake Store API](#)) que proporcione datos de productos.
2. **Obtener los datos:** Usa `fetch()` para hacer una solicitud a la API y obtener la información necesaria.
3. **Mostrar los datos:** Presenta la información en tu proyecto, como una lista de productos, usuarios, etc.
4. **Manejo de errores:** Usa `.catch()` para capturar errores y mostrar un mensaje de error adecuado.
5. **Integración opcional:** Relacioná los datos con una funcionalidad, como un carrito de compras o una lista de favoritos.

Tips claves:

- **Selección de API:** Busca una API pública que tenga los datos que querés incluir en tu proyecto. Algunas opciones son APIs de productos, películas, personas, etc.
- **Manejo de errores:** Usa `.catch()` para capturar cualquier error en la solicitud y mostrar un mensaje de error en la página.
- **Integración con el proyecto:** Pensá cómo podés integrar los datos obtenidos con otras funcionalidades de tu proyecto.

Ejercicio Práctico #2

Obligatorio

Pasos a seguir:

Tips claves:

Crear un carrito de compras dinámico que permita agregar productos de API externa

- Utilizá **fetch()** para obtener una lista de productos desde una API (puede ser la misma API de productos del Ejercicio 1).
- Mostrá los productos en la página en forma de tarjetas o lista.
- Agregá un botón "Añadir al carrito" para cada producto. Al hacer clic en el botón, el producto debe añadirse al carrito.
- Usá **LocalStorage** para almacenar los productos que se agreguen al carrito, de manera que si recarga la página, los productos sigan allí.
- Mostrá la cantidad de productos que hay en el carrito en todo momento, actualizándose cada vez que se añada un nuevo producto.

- **Manipulación del DOM:** Usá métodos como `createElement()` y `appendChild()` para crear dinámicamente las tarjetas de productos y los botones.
- **LocalStorage:** Almacená los productos agregados en LocalStorage utilizando `JSON.stringify()` y `JSON.parse()` para convertir los objetos a formato JSON y viceversa.
- **Actualización del carrito:** Cada vez que un producto sea añadido al carrito, asegurate de actualizar el contador del carrito para poder ver cuántos productos tiene.



¡Nuevo cuestionario en Campus!



 No olvides que los cuestionarios son de carácter obligatorio para poder avanzar con la cursada.